

Fine-Tuning LLMs for Conversational Text-to-SQL

Vitale Ciro

Natural Language Processing
Università degli Studi di Salerno

Luglio 24, 2025

Abstract

Il presente lavoro descrive l'applicazione del *Supervised Fine-Tuning (SFT)* a un *Large Language Model (LLM) open source* per il task di traduzione automatica dal linguaggio naturale a SQL in contesto conversazionale (*Text-to-SQL multi-turn*). A tal fine è stato utilizzato il dataset *CoSQL*, un corpus etichettato di dialoghi *multi-turn* utente-sistema, per addestrare, mediante tecniche di *Parameter-Efficient Fine-Tuning (PEFT)* e in particolare *Low-Rank Adaptation (LoRA)*, il modello *deepseek-coder-1.3B-instruct*, un LLM specializzato nella generazione di codice, composto da 1.3 miliardi di parametri.

L'obiettivo è realizzare un agente conversazionale capace di comprendere richieste in linguaggio naturale e generare query SQL sintatticamente corrette e semanticamente appropriate, adattandole dinamicamente al contesto del dialogo. Il sistema è stato validato tramite un caso di studio in ambito sanitario, riguardante la gestione delle prenotazioni mediche, dimostrando la trasferibilità del modello a domini strutturati.

<https://github.com/cirovitale/text2sql>

Contents

1	Introduzione	3
2	Contesto teorico e lavori correlati	4
2.1	Supervised Fine-Tuning (SFT)	4
2.2	Prompt Engineering	4
2.3	Schema Linking	4
3	Dataset Individuati	5
3.1	Dataset CoSQL	5
4	LLM Open Source	6
4.1	Deepseek Coder 1.3B Instruct	6
5	Prompt	7
5.1	Tecniche di Prompt Engineering	7
5.2	Preprocessing Dataset per Prompt	8
5.3	Struttura Prompt	8
6	Fine-tuning con Low-Rank Adaptation (LoRA)	9
6.1	Configurazione LoRA applicata	10
6.2	Configurazione Parametri Training	10
7	Inferenza	11
7.1	Configurazione Generazione	11
8	Tecnologie utilizzate	11
9	Metriche di Valutazione	12
10	Risultati	13
10.1	Andamento Training e Selezione Modello	13
10.2	Valutazione su CoSQL	14
10.3	Conclusioni Risultati	15
11	Caso di Studio: Prenotazioni Visite Mediche	15
11.1	Motivazioni	15
11.2	Database Schema	16
11.3	Dialogo	17
11.4	Conclusioni Caso di Studio	18
12	Sfide e Limiti	18
13	Sviluppi Futuri	18
14	Conclusioni	19

1 Introduzione

Il task di **Text-to-SQL** rappresenta una delle sfide più significative nell'ambito del *Natural Language Processing (NLP)*, richiedendo la traduzione di domande espresse in linguaggio naturale in query SQL strutturate, capaci di recuperare le informazioni richieste da un database relazionale [1].

I database relazionali sono la tipologia di database più utilizzati al mondo [2], in una vasta gamma di applicazioni da quelle mediche [3, 4] a quelle finanziarie [4, 5]. Il limite di queste strutture dati è che per accedere ai dati in esse contenuti c'è bisogno di effettuare query in *linguaggio SQL*, tanto potenti quanto complesse [4, 6].

Pertanto, un focus di ricerca è su **Natural Language (NL) Interfaces for Databases (NLIBDs)** che consente, a partire da *Natural Language Query (NLQ)*, di effettuare interrogazioni ai database e ricevere i risultati [6, 7].

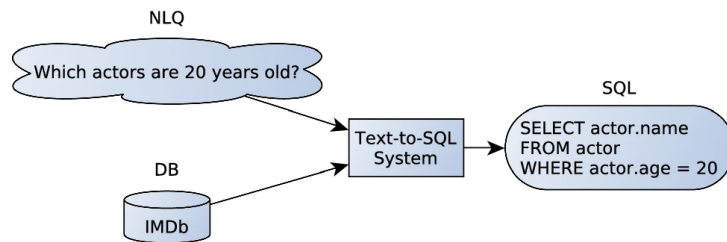


Figure 1: Text-to-SQL Task [6]

Il task **Text-to-SQL** è particolarmente complesso in quanto richiede:

- Comprensione semantica del linguaggio naturale, *intrinsecamente ambiguo* [6, 7].
- Conoscenza e rappresentazione della struttura del database [6, 7].
- Capacità di mappatura tra concetti linguistici e costrutti SQL [6, 7].
- Gestione del contesto, in situazioni *conversazionali* [6, 7].

Negli ultimi anni sono stati introdotti numerosi dataset e modelli per questo task, in particolare il dataset Spider [8] ha posto le basi per un nuovo scenario di riferimento: la generazione di query SQL complesse su domini (*schema*¹) mai visti in addestramento, mettendo alla prova la capacità di *generalizzazione cross-dominio* dei modelli. I progressi nella generazione di query SQL in **contesti single-turn** (dove ogni domanda è indipendente) sono stati significativi. Tuttavia, gli utenti che interagiscono con i sistemi lo fanno in maniera **conversazionale**, definendo la query in più passi, definendo in questo modo il task di generazione **multi-turn**, in cui le domande sono collegate tra loro e il sistema deve essere in grado di formulare la query SQL a partire da un contesto conversazionale [10, 11], pertanto sono stati introdotti dataset come CoSQL [11].

In questo lavoro è stato affrontato il problema di traduzione **Text-to-SQL multi-turn**, mediante il **fine-tuning** con la tecnica **Low-Rank Adaptation (LoRA)** [12], di **deepseek-coder-1.3B-instruct**² [13], la versione più piccola degli LLMs (*Open Source*)

¹Un *database schema* definisce in che modo i dati vengono organizzati all'interno di un database relazionale, includendo vincoli come nomi di tabelle e campi, tipi di dati e le relazioni tra le entità [9].

²<https://huggingface.co/deepseek-ai/deepseek-coder-1.3B-instruct>

orientati al coding di DeepSeek³ da 1,3 miliardi di parametri, utilizzando il dataset CoSQL [11]. La scelta di un modello orientato al codice è motivata dall'intuizione che la capacità di generare sintassi formali sia cruciale per ottenere output sintatticamente e semanticamente corretti in questo task. Il dataset **COSQL** è composto di esempi di *dialoghi utente-sistema* in cui ad ogni domanda dell'utente è associata una query SQL corretta sul database in questione, è un dataset sfidante poiché contiene *domande ambigue*.

2 Contesto teorico e lavori correlati

Il problema della traduzione di linguaggio naturale in query su database è tradizionalmente affrontato come compito di **Semantic Parsing (SP)** [14], dove l'obiettivo è mappare una domanda in un codice eseguibile (in questo caso, SQL). **Approcci classici basati su grammatiche** e regole, che hanno rappresentato storicamente le prime soluzioni per questo task, hanno ottenuto risultati limitati, a causa dell'ambiguità e varietà del linguaggio umano e della necessità di generalizzare a schemi di database diversi [7, 15, 16, 17, 6]. Con l'avvento dei **modelli di Deep Learning**, come *LSTM* e *Transformer*, e dei **Pre-trained Language Model (PLM)**, le prestazioni sul task *Text-to-SQL* sono notevolmente migliorate, purché con alcuni limiti [7]. Una nuova frontiera di ricerca, che è quella esplorata in questo lavoro, è emersa con l'avvento dei **Large Language Model (LLM)** e ha portato all'implementazione di **approcci LLM-based**, che utilizzando tecniche di **fine-tuning**, **prompt engineering** [18, 19, 20, 21, 22, 23, 24] e tecniche di **schema linking** [6, 7, 25] per includere la conoscenza dello schema della struttura database, permette di infrangere i limiti riscontrati con le tecniche precedentemente descritte [7].

2.1 Supervised Fine-Tuning (SFT)

La tecnica più comune riscontrata in letteratura per sviluppare sistemi che effettuino la traduzione da linguaggio naturale a SQL è il **Supervised Fine-Tuning (SFT)** [7], una tecnica di addestramento supervisionato su richieste testuali etichettate con l'SQL atteso. Dato un corpus etichettato, un modello viene *fine-tunato* minimizzando l'errore tra la query SQL predetta e quella di riferimento (*gold*). Nel contesto *multi-turn*, il *SFT* può essere esteso concatenando l'intera *storia del dialogo* come input contestuale al modello ad ogni *turn* (vedi Sezione 5), così che esso impari a **tener conto del contesto passato** nella generazione della query corrente.

2.2 Prompt Engineering

Il **Prompt Engineering** consiste in tecniche di ottimizzazione delle performance degli LLMs attraverso il miglioramento della richiesta testuale [18, 19, 20, 21, 22, 23, 24].

2.3 Schema Linking

Lo **Schema Linking** è il processo di identificazione e associazione tra gli elementi di una NLQ e i componenti di uno schema di database, tipicamente tabelle, colonne e valori [6],

³<https://github.com/deepseek-ai/deepseek-coder>

al fine di fornire un contesto semantico coerente per la generazione della query SQL al modello utilizzato [6, 7, 25].

3 Dataset Individuati

Di seguito sono esaminati dapprima i dataset *single-turn* sul task in questione, focalizzati sulla traduzione in ambito *cross-domain*, e successivamente quelli focalizzati *multi-turn*:

- **Spider** [8]: Dataset *cross-domain* e *single-turn* (ogni domanda è indipendente), composto da 10.181 domande in linguaggio naturale, annotate con 5.693 *unique SQL query*, su un insieme di 200 database su 138 differenti domini [8].
- **SParC (Semantic Parsing in Context)** [10]: Dataset *cross-domain* e *multi-turn*, composto da 4.298 sequenze di domande per un totale di 12.726 domande annotate con le relative query SQL, su un insieme di 200 database su 138 differenti domini (gli stessi di *Spider*) [10].
- **CoSQL (Conversational text-to-SQL) - Utilizzato** [11]: Dataset *cross-domain* e *multi-turn*, composto da 3.007 *dialoghi completi utente-sistema* per un totale di 30.000+ *turns* e 10.000+ query annotate [11], su un insieme di 200 database su 138 differenti domini (gli stessi di *Spider* e *SParC*).

	CoSQL [11]	SParC [10]	Spider [8]
# Q sequence	3.007	4.298	N/A
# user questions	15.598	12.726	10.181
# databases	200	200	200
# domain	138	138	138
# tables	1.020	1.020	1.020
Avg. Q len	11.2	8.1	13
Vocab	9.585	3.794	<i>non specificato</i>
Avg. # Q turns	5.2	3.0	N/A
Unanswerable Q	✓	✗	✗
User intent	✓	✗	✗
System response	✓	✗	✗

Table 1: Confronto tra i Dataset Individuati [8, 10, 11]

3.1 Dataset CoSQL

Il dataset **CoSQL** è stato selezionato per il suo carattere *conversazionale*, *cross-domain* e *multi-turn*, che lo rende particolarmente adatto a task di *Text-to-SQL* in scenari realistici. Oltre alla varietà di domini, i dialoghi includono aspetti linguistici complessi come **ambiguità**, *correzioni* e *fallimenti comunicativi*, riflettendo le dinamiche tipiche delle interazioni umane [11].

CoSQL comprende 3.007 *dialoghi completi utente-sistema* raccolti con protocollo *Wizard-of-Oz* [26], per un totale di 30.000+ *turns* e 10.000+ query annotate, interrogando 200 database appartenenti a 138 domini diversi [11]. Il dataset è *splittato* in *train*, *dev* e *test set* come segue:

	Train	Dev	Test
# Dialogs	2.164	292	551
# Databases	140	20	40

Table 2: CoSQL Dataset Split Statistics [11]

Il dataset è strutturato in tre principali task:

- **SQL-Grounded Dialogue State Tracking - Utilizzato:** Data una conversazione e lo schema del database, consiste nella generazione di query SQL corrispondente, assumendo che la richiesta dell’utente sia interpretabile in linguaggio SQL [11].
- **Response Generation from SQL and Query Results:** Consiste nel generare una risposta in linguaggio naturale che spieghi la query SQL prodotta e i risultati ottenuti dalla sua esecuzione [11].
- **User Dialogue Act Prediction:** Il modello deve determinare, per ogni *turn* dell’utente, se la sua richiesta può essere tradotta in SQL (*label: INFORM_SQL*) o se è necessaria un’altra azione del sistema come una *richiesta di chiarimento* [11].

4 LLM Open Source

In questo lavoro è stato scelto di partire da **LLMs Open Source pre-addestrati per generazione di codice** e istruzioni *relativamente piccoli*, e di **fine-tunarli** sul dataset **CoSQL** (vedi Sezione 3.1) per il task **SQL-Grounded Dialogue State Tracking**.

I motivi di questa scelta sono diversi: da un lato, i modelli addestrati su dati di codice tendono a essere molto abili nel produrre sintassi formalmente corretta (quindi adatti a scrivere SQL validi), e l’ulteriore fine-tuning su istruzioni SQL mediante SFT (vedi sezione 2.1), a partire da richieste in linguaggio naturale, li rende capaci di seguire descrizioni testuali del problema e le rappresentazioni dello schema considerato; dall’altro, utilizzando modello open-source relativamente piccoli (1, 3 miliardi di parametri) consente di effettuare il fine-tuning con risorse computazionali modeste, grazie anche a tecniche di fine-tuning efficienti come LoRA (vedi Sezione 6).

In particolare sono stati considerati: il modello **deepseek-coder-1.3B-instruct** [13], la versione più piccola dei modelli orientati al coding di *DeepSeek* da 1,3 miliardi di parametri, e il modello **Qwen2.5-Coder-1.5B-Instruct** [27] dell’azienda *Qwen*, da 1,5 miliardi di parametri.

4.1 Deepseek Coder 1.3B Instruct

Dopo un’attenta valutazione empirica, molteplici addestramenti e ottimizzazione di parametri, il modello selezionato, che ha raggiunto i migliori risultati (vedi Sezione 10), è stato **deepseek-coder-1.3B-instruct** [13], si tratta di un *modello auto-regressivo* di 1.3 miliardi di parametri specializzato nella generazione di codice, e rappresenta il modello più piccolo orientato al coding rilasciata dall’azienda DeepSeek.

In particolare, questo modello è stato pre-addestrato su un corpus di 2 *triloni* di token, di cui l'87% costituito da codice sorgente, il 10% di materiale in inglese e il 3% in cinese; la finestra di contesto supportata è di 16k token [13].

I vantaggi attesi dall'utilizzo di questo modello per *Text-to-SQL* sono:

- **Generazione codice formalmente corretto:** Avendo visto enormi quantità di codice, il modello conosce potenzialmente bene la grammatica SQL, ciò aiuta a evitare nativamente errori nella formattazione delle query.
- **Comprensione di istruzioni:** La versione instruct è in grado di interpretare un prompt complesso con contesto testuale, come richiesta, e rispondere appropriatamente sottoforma di assistente.
- **Efficienza di fine-tuning:** Il modello, essendo relativamente piccolo, permette tempi di addestramento e requisiti di hardware contenuti, applicando inoltre metodologie ottimizzate come *PEFT/LoRA*.

5 Prompt

Il dataset CoSQL fornisce i dialoghi in forma strutturata (vedi Sezione 3.1), etichettati dalla query corrispondente alla richiesta effettuata, in linguaggio naturale. Per utilizzarli nel fine-tuning effettuato mediante SFT (vedi Sezione 2.1), è stato necessario convertire ogni *turn*, e i corrispondenti output attesi (*query SQL*) in una sequenza di input testuale (vedi Sezione 5.2).

Si noti che nel dataset CoSQL [11] le domande sono presenti in **lingua inglese** e anche il modello `deepseek-coder-1.3B-instruct` [13] è stato addestrato su corpus prevalentemente in lingua inglese, pertanto è stata mantenuta questa lingua anche nella composizione del prompt, effettuando il fine-tuning in lingua originale al fine di non introdurre rumore dato dalle traduzioni.

5.1 Tecniche di Prompt Engineering

Nella composizione del *prompt* sono state utilizzate le seguenti tecniche di **prompt engineering** (vedi Sezione 2.2):

- **Zero-Shot Text-to-SQL:** Non sono forniti esempi diretti di composizione della query, ma solo le *istruzioni*, la *storia del dialogo*, la *question* oppure *query corrente* e il *database schema* [18, 21].
- **Database Schema** [18]: Il database schema è stato fornito includendo i vari `CREATE TABLE` [22].
- **Persona:** Rappresenta la richiesta di impersonificazione di una figura, “*You are a helpful SQL assistant that...*” [23].
- **Delimiter-Pairs:** Sequenza di caratteri utilizzata per separare le diverse sezioni o parti del prompt, è stato utilizzato il seguente stile “**### SEZIONE ###**” [21, 24].

5.2 Preprocessing Dataset per Prompt

Gli *example di training e validation* sono stati generati solo dai *turn* in cui è presente effettivamente una query SQL (ovvero quelli in cui l'utente fa una *domanda answerable* e dopo eventuali chiarimenti si produce una query). Le *user question* in CoSQL sono 15.598 (vedi Sezione 3.1) e per ognuno di essi è stato costruito il prompt con tutta la conversazione fino a quel *turn*. Se un dialogo inizia subito con una *domanda answerable*, allora la *storia del dialogo* sarà vuota prima di essa, se invece c'erano già scambi questi vengono inclusi. Di conseguenza, la lunghezza dei prompt varia, in CoSQL l'80% dei dialoghi ha più di 8 *turn* [11], nel *preprocessing* sono stati filtrati e considerati solo i primi 5 *turn* per ogni dialogo, per ottimizzare delle performance vista la capacità computazionale limitata dall'*hardware* (vedi Sezione 8). Si noti che viene creato un example per ognuno dei 5 *turn*, considerando la storia passata ad ogni step.

Infine, è stata inclusa nel prompt di training la **query corretta** configurando la **attention mask** in modo che la *loss di training e validation* sia calcolata solo sui token della query non vedendoli a priori, definendo per ogni caso le rispettive *labels*, considerando anche la natura *decoder-only* del LLM considerato (vedi Sezione 4.1) [13].

5.3 Struttura Prompt

La **struttura del prompt** deve contenere tutto il contesto necessario a generare la query SQL per la domanda corrente, il prompt utilizzato include:

- **System Instruction:** Rappresenta prompt globale che guida l'intera generazione definendo il task da eseguire, esso stabilisce il ruolo del sistema, i vincoli di output e il tipo di interazione prevista.

“You are a helpful SQL assistant that generates precise SQL queries based on database schemas and natural language questions. Generate only the SQL query without explanations.”

- **Schema** (vedi Sezione 2.3): vengono elencati i nomi e le colonne delle tabelle, fornendo informazioni relativa alla chiave primaria, la tipologia di dato ed eventuali chiavi esterne.
- **Dialogue:** La *storia del dialogo* composta da domande testuali, le relative risposte (query SQL) e la **domanda utente corrente** per la quale il modello deve produrre la query SQL.
- **Token di inizio:** Segnale di inizio sottoforma di rappresentazione *plain text* (per semplicità) per l'output atteso, “SQL:”.

Di seguito il prompt definito:

```
### SYSTEM INSTRUCTIONS ###
```

```
You are a helpful SQL assistant that generates precise SQL queries  
based on database schemas and natural language questions. Generate  
only the SQL query without explanations.
```

```
### SCHEMA ###
```

```
{schema}
```



```
### DIALOGUE ###
{dialogue}
```

SQL:

Con tale approccio il modello in fase di **training** imparerà, a partire da *system instruction*, *schema*, *dialogue* e *richiesta corrente*, a generare la *query SQL* e, in fase di **inferenza**, a partire da *system instruction*, *schema* e *richiesta testuale*, considerando la *conversazione in corso* predirà la *query SQL*.

6 Fine-tuning con Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) è una tecnica che congela i pesi del pre-trained model e introduce un numero esiguo di parametri addizionali, consentendo l'adattamento degli LLMs in maniera efficiente, senza il bisogno di riaddestrare completamente tutti i pesi del modello [12]. Questo approccio riduce significativamente il numero di parametri aggiornati durante il fine-tuning e consente un'efficace personalizzazione anche in scenari con risorse computazionali limitate.

Il modello è stato fine-tunato con la tecnica LoRA in combinazione con la tecnica *SFT* (vedi Sezione 2.1). In pratica, per ogni matrice di pesi W_0 di determinati layer del modello, LoRA introduce una variazione dei pesi ΔW tramite una decomposizione a rango ridotto:

$$\Delta W = BA \quad [12] \tag{1}$$

dove:

- $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$: matrici di rango minore apprese durante il fine-tuning.
- $d \times k$: dimensione del peso originale.
- $r \ll \min(d, k)$: iperparametro chiamato *rango* (rank), che quantifica il numero di parametri aggiunti.
- $W_0 \in \mathbb{R}^{d \times k}$ è la matrice dei pesi originaria (congelata).

Durante il fine-tuning, W_0 rimane congelata e solo le matrici A e B sono soggette ad aggiornamento. Il passaggio forward, per $h = W_0x$, diventa:

$$h = W_0x + \Delta Wx = W_0x + BAx \quad [12] \tag{2}$$

All'inizio dell'addestramento, per garantire che $\Delta W = BA$ non alteri il comportamento del modello pre-addestrato, le matrici sono inizializzate nel seguente modo:

- B : inizializzata a zero [12].
- A : inizializzata con valori casuali estratti da una distribuzione gaussiana [12].

Per migliorare la stabilità e la scalabilità del fine-tuning con LoRA, il termine ΔW viene moltiplicato per un fattore di scalatura $\frac{\alpha}{r}$ [12]:

$$\Delta W = \frac{\alpha}{r}(BA) \tag{3}$$

dove:

- α è un iperparametro scalare (tipicamente $\alpha = r$ oppure $\alpha = 2r$).

Questo consente di evitare il riadattamento manuale del *learning rate* per differenti valori di *rango* r , rendendo la tecnica più robusta a variazioni degli iperparametri [12, 28].

6.1 Configurazione LoRA applicata

Il modello migliore *fine-tunato* è stato addestrato con la seguente **configurazione di parametri** LoRA:

- **Moduli target:**
 - W_q, W_k, W_v di ogni testa di self-attention (*query, key e value projection matrices*).
 - W_o , proiezione dell'output dell'attenzione (che aggrega le teste).
- **Rank** $r = 8$, rappresenta un compromesso comune tra valori più piccoli che riducono ulteriormente i parametri ma possono limitare la capacità di apprendimento delle differenze e valori più alti che rischiano di sovradattare i dati a disposizione per un numero di parametri troppo ampio.
- **Fattore di scala** $\alpha = 16$. Quindi $\frac{\alpha}{r} = 2$, questo implica che le modifiche introdotte da LoRA non vengono né attenuate né amplificate in modo eccessivo, ma ricevono comunque un peso rilevante durante l'ottimizzazione..
- **Dropout** $p = 0.1$, si tratta di una probabilità p che, durante il training, le attivazioni vengano annullate, al fine di *regolarizzare* le modifiche apprese. Con il valore $p = 0.1$ significa che ad ogni passo, c'è il 10% di possibilità di azzerare l'aggiornamento LoRA per un determinato peso.
- **Bias** non addestrati.

6.2 Configurazione Parametri Training

Sono stati configurati gli iperparametri per l'ottimizzazione del training come segue:

- **Batch size:** è stato utilizzato un *batch size* di 32 esempi.
- **Epochs:** Il modello è stato fine-tunato per 2 *epoche complete* per prevenire problematiche come l'overfitting sul dataset non estremamente ampio.
- **Learning rate:** Il *learning rate* iniziale scelto per gli adapter LoRA è $2 \times 10^{-5} = 0,00002$ (2e-5), non troppo alto per evitare di divergere, né troppo basso da richiedere troppe epoche. È stato utilizzato un **linear schedule with warmup**, con warmup per i primi 200 step, quindi il *learning rate* cresce linearmente da 0 a 2e-5 nei primi 200 step. Questo aiuta a iniziare in modo stabile e a convergere meglio.
- Numero di **step di aggiornamento accumulati** prima di eseguire un *backward pass* impostato a 8. Questa tecnica permette di simulare un batch più grande.
- **Logging:** I *log di addestramento* vengono generati ogni 50 step e inviati a **TensorBoard** per monitorare l'andamento del training.

- **Validation:** Il modello viene validato sul *validation set* ogni 100 step.
- **Checkpoint:** Il modello viene salvato ogni 250 step, mantenendo al massimo 2 checkpoint per contenere lo spazio su disco.
- **Mixed Precision:** Non è stato utilizzato il mixed precision training (`fp16 = False`).
- **Lunghezza massima input e output:** È stata impostata una lunghezza massima di sequenza pari a 650 token (vedi Sezione 5.3).

7 Inferenza

Il processo di inferenza è stato implementato con l'obiettivo di evitare risposte incomplete o ambigue, effettuando la **configurazione dei parametri di generazione** e introducendo uno **stopping criteria personalizzato**, al fine di garantire che venga restituita esattamente una singola query SQL corretta e coerente e ottimizzare, anche in *fase di inferenza* la generazione auto-regressiva del modello già fine-tunato sul task specifico.

7.1 Configurazione Generazione

I parametri adottati per la generazione del testo sono stati selezionati per garantire un equilibrio tra **determinismo** e **varietà** nelle risposte generate:

- **Temperature = 0.3:** Una *temperature* più bassa aumenta il “*determinismo*” del modello, riducendo la probabilità di generare token casuali.
- **Top-p = 0.8:** Questo parametro controlla il *sampling*, limitando la generazione ai token che cumulano almeno l'80% della probabilità.

In combinazione con una temperatura bassa, consente di mantenere la diversità quando esistono più soluzioni corrette, senza sacrificare coerenza e correttezza.

- **Stopping Criteria:** Meccanismo di **interruzione personalizzata** della generazione in presenza di simboli o pattern sintattici specifici, garantendo che venga prodotta esattamente una query SQL. Esso è basato sulla rilevazione di *sequenze sintattiche comuni di fine query SQL*, come il punto e virgola “;” e pattern testuali come “Q:” (*Question*) o “A:” (*Answer*) che segnalano un cambio di *turn* nel dialogo.
- **Max new tokens = 150:** lunghezza massima dei token generati, sufficiente per query SQL complesse.

8 Tecnologie utilizzate

L'implementazione è stata realizzata completamente in **Python** mediante le librerie dell'ecosistema **Hugging Face** e **PyTorch**. Di seguito i principali componenti tecnologici:

- **Libreria Transformers di Hugging Face:**

- **Caricamento Modello:** L'interfaccia `AutoModelForCausalLM` per caricare *deepseek-coder-1.3B-instruct* dal repository HuggingFace, insieme al corrispondente tokenizer (`AutoTokenizer`).
- **Training:** La libreria `Transformers` fornisce le funzionalità necessarie per il **training auto-regressivo** dei modelli generativi, tra cui `Trainer` e `TrainingArguments`, inclusa l'implementazione integrata del **masking causale** e **deltenzione** e il calcolo della loss function di **Cross Entropy**:

$$\mathcal{L}_{CE} = -\frac{1}{T} \sum_{t=1}^T \log P(x_t | x_1, \dots, x_{t-1}) \quad (4)$$

Dove:

- * T : lunghezza sequenza.
- * x_t : token in posizione t .
- * $P(x_t | x_1, \dots, x_{t-1})$: probabilità del token x_t dato il contesto precedente.

Inoltre, essa supporta pienamente l'integrazione con la libreria `PEFT` per il fine-tuning efficiente.

- **Inferenza:** La **generazione auto-regressiva** in fase di inferenza è stata configurata con gli opportuni parametri mediante le interfacce `AutoModelForCausalLM` e `PeftModel`.
- **Libreria PEFT e LoRA** utilizzata per applicare il fine-tuning solo a una frazione dei parametri del modello, consente di definire un oggetto `LoraConfig` con i parametri desiderati (rango, α , dropout, moduli target, bias e tipologia di task).
- **Libreria PyTorch.**
- **TensorBoard** utilizzata per la **visualizzazione interattiva** dell'andamento del *training*, monitorando in tempo reale e facilitando l'identificazione di *overfitting* o *instabilità* del processo.
- **Hardware Utilizzato**, il fine-tuning è stato eseguito su una macchina dotata di:
 - **Processore:** *Intel 11th Gen i5-11400F 2.60GHz*.
 - **Memoria:** *16GB*.
 - **GPU:** *NVIDIA GeForce GTX 1060 6GB*.

9 Metriche di Valutazione

La metrica di valutazione maggiormente utilizzata per il task **Text-to-SQL** è **Exact Set Match** [8, 10], in cui una predizione è considerata corretta se il *set di componenti della query SQL* coincide esattamente con la *query gold* (effettivamente corretta) [6]. Questa metrica è stata calcolata nei seguenti modi:

- **Question Match** [10]: Punteggio di exact set matching calcolato su ciascuna singola domanda. Per ogni domanda vale 1 se e solo se *tutte* le clausole SQL (`SELECT`, operatori di aggregazione, ecc.) previste coincidono esattamente con quelle della query gold, 0 altrimenti [10].

- **Interaction Match** [10]: Punteggio di exact set matching calcolato sull'intera interazione (serie di domande correlate). Per ogni interazione vale 1 se e solo se *ogni domanda* dell'interazione ottiene un *exact set match*, 0 altrimenti [10].

10 Risultati

Di seguito vengono discussi i **risultati** ottenuti **durante il processo di fine-tuning e validazione** del modello selezionato per il task *Text-to-SQL* multiturn. L'analisi si concentra sia sull'andamento delle curve di perdita (*loss*) durante l'addestramento e la validazione, sia sulla valutazione finale tramite le metriche di valutazione (vedi Sezione 9) sul *test set*, confrontando il modello base selezionato e il modello fine-tunato a partire da quest'ultimo.

10.1 Andamento Training e Selezione Modello

Nel corso della sperimentazione sono stati testati diverse configurazioni di parametri di addestramento dei modelli **Qwen2.5-Coder-1.5B-Instruct** e **deepseek-coder-1.3b-instruct**. I risultati di tali esperimenti sono illustrati di seguito (vedi Figura 2), dove è possibile confrontare l'evoluzione della **train/loss** per diversi run. Il modello identificato come **model-007** ha mostrato la convergenza più rapida e stabile, con una *training loss* finale di circa 0.3319 allo step 1.500.

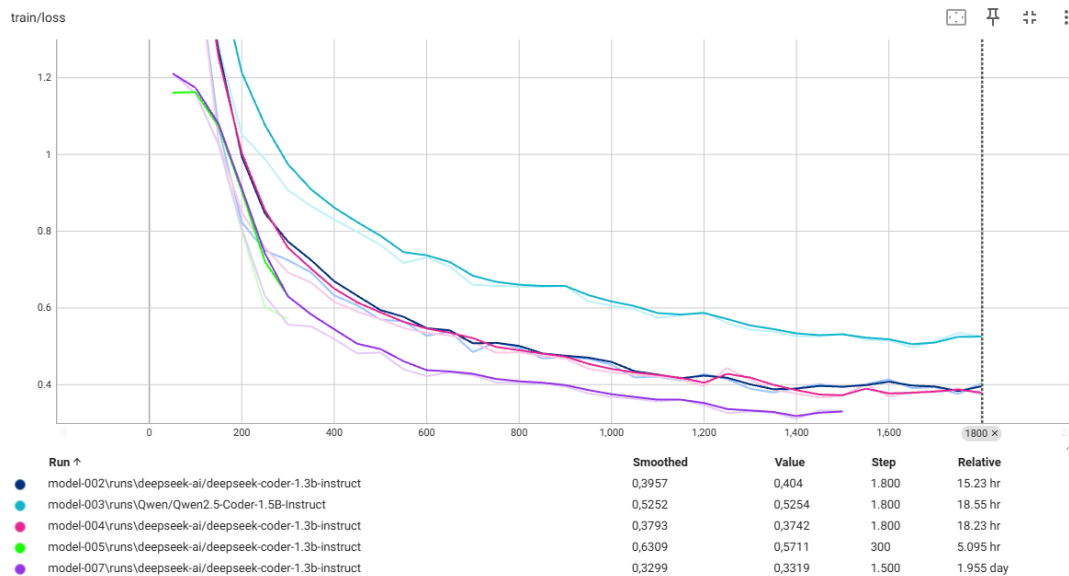


Figure 2: Andamento della **train/loss** nei diversi esperimenti. Il modello 007 mostra la migliore convergenza.

Per evitare fenomeni di **overfitting**, la selezione del checkpoint finale è stata effettuata sulla base della **eval/loss** (vedi Figura 3), oltre che sulla base di **train/loss** descritta in precedenza. Il modello 007 ha mantenuto una validazione stabile per più di **1.000 step**, raggiungendo un valore minimo di *validation loss* pari a 0.5653, dopodiché si è iniziato a intravedere un cambio di direzione a suggerire un principio di *overfitting*, benché minimo. Per tale motivo è stato interrotto l'addestramento intorno allo step 1.500 ed è stato selezionato il modello allo step 1.000 come punto ottimale.

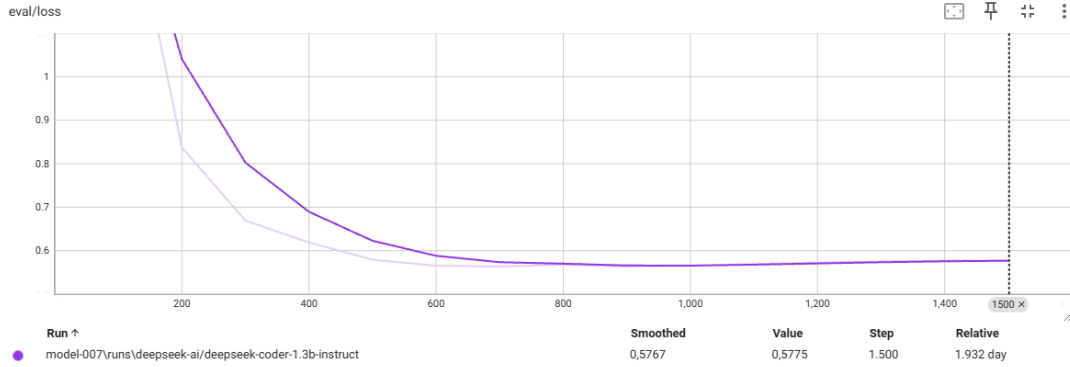


Figure 3: Andamento della *eval/loss* per il modello 007. Il minimo stabile è raggiunto attorno a step 1.000.

10.2 Valutazione su CoSQL

Al termine della fase di addestramento, il modello selezionato (007, basato sul modello “base” `deepseek-coder-1.3b-instruct`) è stato testato su una parte del *test set* di CoSQL. I risultati ottenuti sono confrontati con quelli del modello base non addestrato (vedi Tabella 3).

Modello	Question Match	Interaction Match
Base	4,35% (4/92)	0,00% (0/29)
Fine-tuned	31,52% (29/92)	10,35% (3/29)

Table 3: Confronto delle prestazioni tra *modello base* e *modello fine-tunato* sul test set CoSQL.

Dall’analisi emerge un chiaro miglioramento, il **modello fine-tunato** ha generato più di sette volte il numero di query esatte rispetto al modello base. Inoltre, le interazioni complete corrette dimostrano che il modello inizia a mantenere la coerenza su più *turn*.

Osservando nel dettaglio i risultati, si nota che:

- Il **modello base**, pur avendo ricevuto prompt specifici, spesso **risponde in linguaggio naturale** o con SQL parziali, mentre il **modello fine-tunato** produce **query ben formate e complete** nella maggior parte dei casi.
- In alcuni casi, la metrica di **Exact Set Match** utilizzata (vedi Sezione 9) penalizza ingiustamente differenze superficiali producendo **falsi negativi** [8], come ad esempio:

– Uso di alias sintattici:

`visitor.Name`

invece della gold:

`T2.Name`

- Condizioni più generali ma corrette (anche secondo lo *schema* considerato), come:

```
SELECT * FROM Documents WHERE Document_Name LIKE '%w%'
OR Document_Description LIKE '%w%';
```

invece della gold:

```
SELECT * FROM Documents WHERE Document_Description LIKE '%w%'
```

Per questi motivi, metriche più robuste, come la **Execution Accuracy** [8], che verificano se la query prodotta restituisce lo stesso output sul database della gold query, sarebbero più adatte per una valutazione affidabile e coerente, mitigando il rischio di produrre molti *falsi negativi* che vanno fortemente ad incidere sui risultati del *testing* e, quindi, sulla valutazione del modello risultante.

10.3 Conclusioni Risultati

Nonostante i falsi negativi prodotti, come illustrato precedentemente, i risultati ottenuti dal *modello fine-tunato* dimostrano un effettivo apprendimento del task, anche in condizioni stringenti e un **netto miglioramento** rispetto alle prestazioni del modello *base* (vedi Tabella 3).

11 Caso di Studio: Prenotazioni Visite Mediche

Questo caso di studio è stato concepito per **valutare l'efficacia del modello fine-tunato nel dominio sanitario**, in particolare nell'*ambito di gestione di visite mediche specialistiche attraverso un'interazione conversazionale*. L'obiettivo è analizzare la capacità del modello di generare query SQL coerenti e corrette in uno scenario realistico.

11.1 Motivazioni

Le motivazioni sono molteplici, questi sistemi guidati dal linguaggio naturale, applicati al settore sanitario, possono migliorare significativamente l'accessibilità e l'efficienza nella gestione degli appuntamenti, consentendo a pazienti non esperti di interagire con basi di dati in maniera diretta. Il caso di studio vede l'utilizzo del modello risultante dal lavoro effettuato, che rappresenta un **agente conversazionale** capace di comprendere domande complesse, generare query SQL e aggiornare dinamicamente lo stato dell'interazione.

11.2 Database Schema

Il database progettato per questo scenario è rappresentato dal seguente **schema**, che consente di rappresentare *utenti*, *appuntamenti* e *specialisti*:

```
CREATE TABLE User (  
    id INTEGER PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    surname VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    phone VARCHAR(20)  
);  
  
CREATE TABLE Category (  
    id INTEGER PRIMARY KEY,  
    name VARCHAR(100) NOT NULL CHECK (  
        name IN ('Cardiology', 'Orthopedics', 'Dermatology',  
        'Neurology', 'General Medicine')  
    )  
);  
  
CREATE TABLE Specialist (  
    id INTEGER PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    surname VARCHAR(100) NOT NULL,  
    category_id INTEGER NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    FOREIGN KEY (category_id) REFERENCES Category(id)  
);  
  
CREATE TABLE Appointment (  
    id INTEGER PRIMARY KEY,  
    user_id INTEGER NOT NULL,  
    specialist_id INTEGER NOT NULL,  
    datetime DATETIME NOT NULL,  
    status VARCHAR(20) NOT NULL CHECK (  
        status IN ('active', 'cancelled', 'completed')),  
    FOREIGN KEY (user_id) REFERENCES User(id),  
    FOREIGN KEY (specialist_id) REFERENCES Specialist(id)  
);
```


11.3 Dialogo

Di seguito si riportano i risultati di un dialogo *multi-turn* con il sistema proposto, con relativa generazione di query SQL.

USER: Hello, my name is Ciro and my surname is Vitale.

I would like to know what my next appointment is.

SYSTEM:

```
SELECT * FROM Appointment AS T1
JOIN User AS T2
ON T1.user_id = T2.id
JOIN Specialist AS T3
ON T1.specialist_id = T3.id
WHERE T2.name = "Ciro Vitale";
```

USER: What are the appointments I have deleted?

SYSTEM:

```
SELECT * FROM Appointment AS T1
JOIN User AS T2
ON T1.user_id = T2.id
JOIN Specialist AS T3
ON T1.specialist_id = T3.id
WHERE T2.name = 'Ciro' AND T2.surname = 'Vitale'
AND T1.status = 'cancelled';
```

USER: What is my registered email?

SYSTEM:

```
SELECT email FROM User
WHERE name = "Ciro" AND surname = 'Vitale';
```

USER: List all available specialists in dermatology.

SYSTEM:

```
SELECT name FROM Specialist WHERE category_id =
( SELECT id FROM Category WHERE name = 'Dermatology' )
AND status = 'active';
```

USER: Can I view all my appointments with Dr. Rossi?

SYSTEM:

```
SELECT * FROM Appointment WHERE specialist_id =
( SELECT id FROM Specialist WHERE name = "Dr. Rossi" )
AND status = "active" AND specialist_id =
( SELECT id FROM Specialist WHERE name = "Dr. Rossi" )
AND specialist_id =
( SELECT id FROM Specialist WHERE name = "Dr. Rossi" );
```

USER: What is the total number of appointments by category?

SYSTEM:

```
SELECT Category.name, count ( * ) FROM Appointment
JOIN Category
ON Appointment.specialist_id = Category.id
GROUP BY Category.name;
```

11.4 Conclusioni Caso di Studio

Le generazioni risultanti dal dialogo evidenziano le **buone capacità del modello** nella interpretazione semantica di richieste in linguaggio naturale e nella successiva generazione di query SQL contestualmente appropriate. Nonostante si osservano **occasionalità imprecisioni o piccole ridondanze nelle formulazioni**, il sistema dimostra una **comprensione solida del contesto conversazionale e dello schema del database**, riuscendo a **produrre risposte coerenti e funzionalmente corrette**.

12 Sfide e Limiti

Lo sviluppo di un sistema di *Text-to-SQL* conversazionale come quello presentato comporta numerose sfide e sono stati individuati i seguenti limiti, che in parte sono stati pienamente affrontati nella soluzione proposta:

- **Mantenimento stato e coerenza multi-turn:** Il modello deve ricordare le informazioni già fornite dall'utente o già ottenute dal database, per evitare di ripetere query inutili e per restringere correttamente le interrogazioni successive. Nell'approccio utilizzato, il contesto viene fornito per intero ad ogni *turn* sotto forma di prompt esteso, il che dà al modello tutte le informazioni necessarie.
- **Dimensione dataset considerato:** Il dataset considerato contiene un totale di 31.148 *turn* etichettati il che potrebbe risultare limitante, non riuscendo a rappresentare e, quindi, ad insegnare al modello tutte le sfaccettature di query per il recupero delle informazioni a partire dal linguaggio naturale.
- **Generalizzazione a schemi non visti:** Il modello deve generalizzare la sua comprensione a nuove tabelle e colonne.
- **Robustezza sintattica:** Il modello deve generare query valide sintatticamente.
- **Dimensione modello selezionato:** Un limite intrinseco è la dimensione del modello considerato, che rappresenta il modello più piccolo della famiglia degli *LLM coder* di Deepseek (vedi Sezione 4.1), il che rende il training efficiente, ma pone un limite notevole alle prestazioni.

13 Sviluppi Futuri

Considerando le sfide e i limiti individuati, sono stati definiti i seguenti miglioramenti futuri che si possono provare ad apportare:

- **Integrazione con Database:** Integrare un database SQL per eseguire le query SQL generate.
- **Utilizzo di Dataset Multipli:** Addestrare congiuntamente su Spider [8], SPaRC [10] e CoSQL [11] ed effettuare un *multi-task fine-tuning* (statico e contestuale).
- **Valutazione con Execution Accuracy:** Integrare metriche basate sull'esecuzione della query (Execution Accuracy), che verificano la correttezza del risultato SQL eseguito sul database, superando i limiti dell'*Exact Set Match* su query sintatticamente diverse ma semanticamente equivalenti [8].

- **Parsing Incrementale e Decodifica Auto-Regressiva Vincolata:** Integrare *PICARD* [29], che vincola la generazione token per token a produrre stringhe conformi alla grammatica SQL e allo schema [6].
- **Modello più Grande:** Effettuare il fine-tuning con la stessa logica, adattando la configurazione dei parametri, del miglior modello di Deepseek per la traduzione in codice: *DeepSeek-Coder-V2-Instruct* composto di 236B di parametri [30].
- **Modelli di Reasoning:** Migliorare le capacità di reasoning del modello [31].

14 Conclusioni

Il modello addestrato ha mostrato un netto miglioramento rispetto alla versione base. L’approccio impiegato ha dimostrato l’**efficacia del fine-tuning di modelli di dimensioni molto piccole orientati al coding** anche in presenza di risorse limitate, pur evidenziando alcuni limiti di generazione mediante le metriche calcolate.

Il **caso di studio sulle prenotazioni mediche** conferma la trasferibilità del modello al relativo contesto reale, rendendo il sistema promettente per applicazioni conversazionali in domini strutturati. In particolare, la capacità del modello di generare query SQL corrette e contestualizzate consente agli utenti di interagire in linguaggio naturale ed effettuare interrogazioni dirette su basi di dati relazionali, abilitando scenari di accesso intuitivo e diretto a informazioni complesse.

References

- [1] Z. Cai, X. Li, B. Hui, M. Yang, B. Li, B. Li, Z. Cao, W. Li, F. Huang, L. Si *et al.*, “Star: Sql guided pre-training for context-dependent text-to-sql parsing,” *arXiv preprint arXiv:2210.11888*, 2022.
- [2] db engines.com, “Db-engines ranking - the db-engines ranking ranks database management systems according to their popularity. the ranking is updated monthly.” <https://db-engines.com/en/ranking>, accessed: 2025-07-15.
- [3] R. Hillestad, J. Bigelow, A. Bower, F. Girosi, R. Meili, R. Scoville, and R. Taylor, “Can electronic medical record systems transform health care? potential health benefits, savings, and costs,” *Health affairs*, vol. 24, no. 5, pp. 1103–1117, 2005.
- [4] V. Zhong, C. Xiong, and R. Socher, “Seq2sql: Generating structured queries from natural language using reinforcement learning,” *arXiv preprint arXiv:1709.00103*, 2017.
- [5] T. Beck, A. Demirgüç-Kunt, and R. Levine, “A new database on the structure and development of the financial sector,” *The World Bank Economic Review*, vol. 14, no. 3, pp. 597–605, 2000.
- [6] G. Katsogiannis-Meimarakis and G. Koutrika, “A survey on deep learning approaches for text-to-sql,” *The VLDB Journal*, vol. 32, no. 4, pp. 905–936, 2023.

- [7] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, “Next-generation database interfaces: A survey of llm-based text-to-sql,” *arXiv preprint arXiv:2406.08426*, 2024.
- [8] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task,” *arXiv preprint arXiv:1809.08887*, 2018.
- [9] IBM, “What is a database schema?” <https://www.ibm.com/think/topics/database-schema>, accessed: 2025-07-10.
- [10] T. Yu, R. Zhang, M. Yasunaga, Y. C. Tan, X. V. Lin, S. Li, H. Er, I. Li, B. Pang, T. Chen *et al.*, “Sparc: Cross-domain semantic parsing in context,” *arXiv preprint arXiv:1906.02285*, 2019.
- [11] T. Yu, R. Zhang, H. Y. Er, S. Li, E. Xue, B. Pang, X. V. Lin, Y. C. Tan, T. Shi, Z. Li *et al.*, “Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases,” *arXiv preprint arXiv:1909.05378*, 2019.
- [12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, “Lora: Low-rank adaptation of large language models.” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [13] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, “Deepseek-coder: When the large language model meets programming – the rise of code intelligence,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [14] S. Papicchio, P. Papotti, and L. Cagliero, “Evaluating ambiguous questions in semantic parsing,” in *2024 IEEE 40th International Conference on Data Engineering Workshops (ICDEW)*. IEEE, 2024, pp. 338–342.
- [15] F. Li and H. V. Jagadish, “Constructing an interactive natural language interface for relational databases,” *Proceedings of the VLDB Endowment*, vol. 8, no. 1, pp. 73–84, 2014.
- [16] T. Mahmud, K. A. Hasan, M. Ahmed, and T. H. C. Chak, “A rule based approach for nlp based query processing,” in *2015 2nd international conference on electrical information and communication technologies (EICT)*. IEEE, 2015, pp. 78–82.
- [17] T. Yu, C.-S. Wu, X. V. Lin, B. Wang, Y. C. Tan, X. Yang, D. Radev, R. Socher, and C. Xiong, “Grappa: Grammar-augmented pre-training for table semantic parsing,” *arXiv preprint arXiv:2009.13845*, 2020.
- [18] S. Chang and E. Fosler-Lussier, “How to prompt llms for text-to-sql: A study in zero-shot, single-domain, and cross-domain settings,” *arXiv preprint arXiv:2305.11853*, 2023.
- [19] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, “Text-to-sql empowered by large language models: A benchmark evaluation,” *arXiv preprint arXiv:2308.15363*, 2023.

- [20] Q. Ye, M. Axmed, R. Pryzant, and F. Khani, “Prompt engineering a prompt engineer,” 2024. [Online]. Available: <https://arxiv.org/abs/2311.05661>
- [21] M. Suzgun, L. Melas-Kyriazi, and D. Jurafsky, “Prompt-and-rerank: A method for zero-shot and few-shot arbitrary textual style transfer with small language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.11503>
- [22] N. Rajkumar, R. Li, and D. Bahdanau, “Evaluating the text-to-sql capabilities of large language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.00498>
- [23] J. Kim, N. Yang, and K. Jung, “Persona is a double-edged sword: Mitigating the negative impact of role-playing prompts in zero-shot reasoning tasks,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.08631>
- [24] OpenAI, “Best practices for prompt engineering with the openai api,” help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api, accessed: 2025-07-12.
- [25] K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi, “The death of schema linking? text-to-sql in the age of well-reasoned language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.07702>
- [26] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gašić, “Multiwoz – a large-scale multi-domain wizard-of-oz dataset for task-oriented dialogue modelling,” 2020. [Online]. Available: <https://arxiv.org/abs/1810.00278>
- [27] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, “Qwen2. 5-coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [28] G. Yang and E. J. Hu, “Feature learning in infinite-width neural networks,” *arXiv preprint arXiv:2011.14522*, 2020.
- [29] T. Scholak, N. Schucher, and D. Bahdanau, “Picard: Parsing incrementally for constrained auto-regressive decoding from language models,” *arXiv preprint arXiv:2109.05093*, 2021.
- [30] Q. Zhu, D. Guo, Z. Shao, D. Yang, P. Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma *et al.*, “Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence,” *arXiv preprint arXiv:2406.11931*, 2024.
- [31] S. Papicchio, S. Rossi, L. Cagliero, and P. Papotti, “Think2sql: Reinforce llm reasoning capabilities for text2sql,” 04 2025.